

Módulo 1 y 2: Entorno y herramientas / Fundamentos del lenguaje

Contenidos clave

Módulo 1

Instalación: Python 3.12, Ambientes virtuales / Poetry: Python 3.12 es la versión más reciente del lenguaje. Los ambientes virtuales (venv, virtualenv) y gestores como Poetry permiten aislar dependencias de cada proyecto. Se crea un entorno virtual y dentro instalas librerías sin afectar tu sistema global; sirve para evitar conflictos entre proyectos.

IDEs: PyCharm y extensiones útiles, VS Code: Programas para ejecutar código con herramientas integradas; ofrecen autocompletado, depuración, integración con Git y aceleran el desarrollo al reducir errores. PyCharm para proyectos grandes con frameworks (Django, Flask y FastAPI), incluye un asistente con IA, es un gestor de proyectos con soporte para requirements.txt y pyproject.toml; por otro lado VS Code para proyectos ligeros con extensiones útiles (Python, Pylance, Jupyter).

Estructura de proyecto con pyproject.toml / requirements.txt: Son archivos que definen dependencias y configuración del proyecto, requirements.txt es una lista de librerías con versiones y pyproject.toml es un estándar moderno que incluye dependencias, configuración de herramientas (Poetry, Black, etc.). Sirven para reproducir el entorno de desarrollo en cualquier máquina.

PEP 8 (estilo) y PEP 20 (Zen de Python); automatización con black, isort, ruff y pre-commit: PEP 8 es una guía de estilo para escribir código limpio y legible. PEP 20 son principios filosóficos del lenguaje para guiar el diseño. Las herramientas:

- Black: formatea código automáticamente.
- Isort: ordena imports.
- Ruff: analiza errores de estilo.
- pre-commit: ejecuta estas herramientas antes de cada commit en Git. aseguran que el código sea consistente y profesional.

Módulo 2

Sintaxis, indentación, variables y alcance: Es la forma en que se escribe el código para que Python lo entienda. Indentación: (espacios al inicio de la línea) define bloques de código, da estructura legibilidad.

Tipos básicos y colecciones (list, dict, set, tuple): Son las estructuras que guardan los datos; tienen que ver con cómo se almacenan y organizan la información.

- list: Las listas son mutables, dinámicas y ordenadas, pueden almacenar elementos de cualquier tipo.
- Dict: Los diccionarios son colecciones de elementos en el que cada uno se compone de una clave y un valor, es mutable, mantiene el orden de inserción y permite acceder a los valores mediante sus claves.

- set: Los conjuntos son una colección desordenada de elementos únicos, no permite duplicados y permite elementos de diferentes tipos.
- tuple: Son colecciones ordenadas e inmutables, ocupan menos memoria que las listas.

Control de flujo (if, switch, for, while) y pattern matching: Son las estructuras que deciden qué parte del código se ejecuta, realizan la lógica y toma de decisiones del programa.

- if: Ejecuta un bloque de código sólo si se cumple una condición específica.
- switch: Esta estructura se puede emular con if-elif, con diccionarios o incluso el operador match. Una manera eficiente de simularlo es usar un diccionario donde las claves representan los casos y los valores son funciones o resultados asociados.
- for: Sirve para iterar sobre los elementos de un objeto iterable como listas, cadenas, diccionarios o rangos, y a diferencia de otros lenguajes, no requiere una variable de control explícita.
- while: Ejecuta un bloque de código repetidamente mientras una condición sea verdadera. Es ideal cuando no se conoce el número de iteraciones.

Pattern matching se refiere a una nueva sintaxis introducida en Python 3.10 con la instrucción match/case, que permite comparar valores y estructuras de datos de forma declarativa, similar a un switch mejorado, y también a la coincidencia de patrones con expresiones regulares.

Errores y excepciones (try-except): Es el mecanismo para manejar errores sin que el programa se detenga, ayuda con la robustez y estabilidad del código. Los errores más frecuentes al usar try-except en Python incluyen capturar excepciones genéricas, no especificar el tipo de error, y manejar mal el flujo de ejecución dentro de los bloques try y except.

- Capturar excepciones genéricas sin especificar el tipo oculta errores inesperados y dificulta la depuración, ya que cualquier excepción será atrapada, incluso aquellas que no se anticipaban, es mejor capturar excepciones específicas.
- El orden incorrecto de los bloques except importa mucho cuando se manejan múltiples excepciones. Python ejecuta la primera cláusula except que coincida; colocar una excepción base antes de una derivada puede hacer que la excepción derivada nunca se capture.
- No usar else y finally adecuadamente. El primero se ejecuta solo si no ocurre ninguna excepción; es útil para código que solo debe ejecutarse cuando el try es exitoso. El segundo, se ejecuta siempre, es útil para cerrar archivos o liberar recursos, incluso si ocurre un error.

Control o gestión de excepciones: Extiende el manejo de errores para registrar, relanzar o limpiar recursos.

Objetivos

Módulo 1

- Instalar/gestionar versiones y entornos

Python 3.12:

Katherine Quiryat Rosales Garduño

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

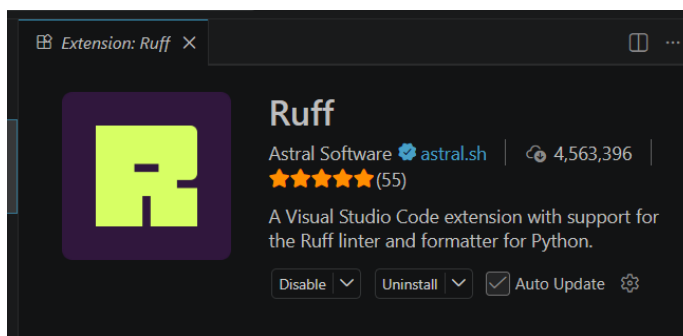
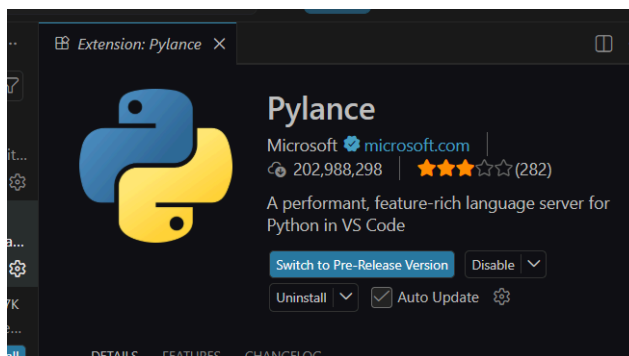
PS C:\Users\Quiryat> py --version
Python 3.14.6
PS C:\Users\Quiryat> |
```

Git:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Quiryat> py --version
Python 3.14.6
PS C:\Users\Quiryat> git --version
git version 2.54.0.windows.1
PS C:\Users\Quiryat> |
```

Extensiones de Visual Studio Code: Python, Pylance y Ruff:



- Configurar IDE y herramientas de calidad

Poetry:

```
Windows PowerShell
PS C:\Users\Quiryat> (Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | py -
Retrieving Poetry metadata

# Welcome to Poetry!

This will download and install the latest version of Poetry,
a dependency and package manager for Python.

It will add the 'poetry' command to Poetry's bin directory, located at:

C:\Users\Quiryat\AppData\Roaming\Python\Scripts

You can uninstall at any time by executing this script with the --uninstall option,
and these changes will be reverted.

Installing Poetry (2.4.3)
Installing Poetry (2.4.3): Creating environment
Installing Poetry (2.4.3): Installing Poetry
Installing Poetry (2.4.3): Creating script
Installing Poetry (2.4.3): Done

Poetry (2.4.3) is installed now. Great!
```

Añadimos Poetry al Path:

```
PS C:\Users\Quiryat> [Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";C:\Users\Quiryat\AppData\Roaming\Python\Scripts", "User")
PS C:\Users\Quiryat> |
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Quiryat> poetry --version
Poetry (version 2.4.3)
PS C:\Users\Quiryat> |
```

- Aplicar PEP 8/PEP 20 y documentar excepciones locales

Las herramientas black, isort y ruff se aplicaron de manera automática cuando ejecutamos el git commit, cumpliendo las reglas de PEP 8 sobre sangría, espacios alrededor de operadores y saltos de línea; el importe de las librerías al inicio del archivo y agrupadas según la convención estándar de PEP 8; y la evaluación de la estructura del código asegurando la ausencia de variables no utilizadas o en formato incorrecto.

Aplicando el PEP 20, se especificó explícitamente el tipo de cada excepción a manejar; cuando el registro 3 presentó un precio inválido el script no falló ni ignoró el evento, sino que emitió un mensaje explicativo y continuó la ejecución; el código se estructuró dividiendo responsabilidades con funciones claras e indentación limpia.

Se documentó el comportamiento de la función definida y el flujo según el tipo exacto de falla, apertura o ausencia del archivo, fallo en la sintaxis del JSON y fallo en datos individuales de un registro.

Módulo 2

- Manejar estructuras de datos y control de flujo

Uso de colecciones, list[dict] definido como tipo de retorno en cargar_y_procesar_datos y acumulador dinámicamente en productos_validos.append(valido); dict pasado como parámetro individual en procesar_registro(item: dict) y retornado como una nueva estructura limpia de diccionario; path para la manipulación de la ruta del archivo mediante la librería pathlib. Control de flujo con el bucle for: for item in datos: Itera secuencialmente la lista de diccionarios cargada desde el JSON.

- Implementar manejo de errores robusto

Uso de una excepción personalizada de dominio, la cual permite diferenciar fallos de lógica de negocio de fallos del sistema o de sintaxis.

```
9 class ErrorDatoInvalido(Exception):
10     """Excepción para errores en campos del JSON."""
11
```

La captura explícita de excepciones.

```
22         if not re.match(PATRON_SKU, sku):
23             raise ErrorDatoInvalido(f"SKU '{sku}' no cumple el formato 'AAA-000'")
24
25         try:
26             precio = float(precio_raw)
27             if precio <= 0:
28                 raise ValueError("El precio debe ser positivo")
29         except (ValueError, TypeError) as e:
30             raise ErrorDatoInvalido(f"Precio inválido ({precio_raw}): {e}")
31
32         return {"sku": sku, "nombre": nombre, "precio": precio}
33
34         case {"activo": False}:
35             raise ErrorDatoInvalido("Registro inactivo")
36
37         case _:
38             raise ErrorDatoInvalido("Estructura de registro no compatible o incompleta")
39
```

- Usar pattern matching en casos adecuados y expresiones regulares

La definición del patrón: PATRON_SKU = r"^[A-Z]{3}-\d{3}\$" exige exactamente 3 letras mayúsculas, un guión y 3 dígitos; la evaluación en el código if not re.match(PATRON_SKU, sku): valida la sintaxis de la cadena antes de aceptarla como válida.

El uso de pattern matching se visualiza al extraer las variables sku, nombre y precio_raw al mismo tiempo que verifica que activo sea true y que sku y nombre sean de tipo str. La coincidencia condicional: case {"activo": False} filtra de inmediato registros desactivados sin necesidad de evaluar el resto de sus llaves.

```
match item:
    case {
        "sku": str(sku),
        "nombre": str(nombre),
        "precio": precio_raw,
        "activo": True,
    }:
```

Laboratorio

Módulo 1

- Crear proyecto con Poetry, activar venv, instalar black/isort/ruff

1. Creamos el repositorio y clonamos en local:

```
PS C:\Users\Quiryat\Documents> cd Python
PS C:\Users\Quiryat\Documents\Python> ls
PS C:\Users\Quiryat\Documents\Python> git clone https://github.com/quiryat09S/python-mastery-capstone.git
Cloning into 'python-mastery-capstone'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
PS C:\Users\Quiryat\Documents\Python> ls

Directorio: C:\Users\Quiryat\Documents\Python

Mode                LastWriteTime         Length Name
----                -
d-----          11/09/2026  12:16 p. m.         python-mastery-capstone
```

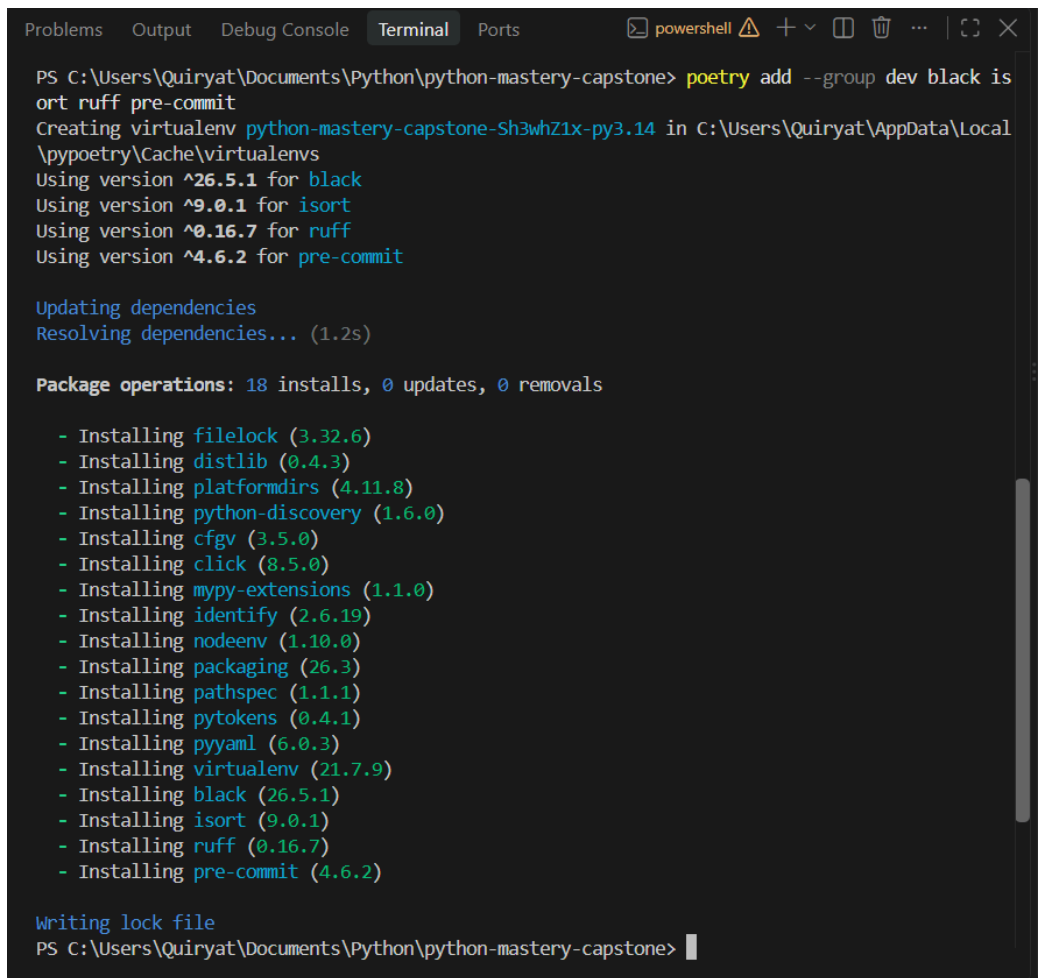
2. Inicializamos la estructura base del proyecto (archivo base) para gestionar dependencias y empaquetado mediante la especificación de pyproject.toml:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry init -n
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> █
```

Se genera el archivo pyproject.toml en la raíz del repositorio:



3. Entorno virtual (activar venv): Poetry gestiona automáticamente la creación del entorno virtual dentro de su caché de entornos local.
4. Instalación de black/isort/ruff: Incorporamos al entorno de desarrollo las herramientas automáticas de formateo, ordenamiento de importaciones y lintado de código estático (siguiendo los estándares PEP 8 y PEP 20).



```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry add --group dev black isort ruff pre-commit
Creating virtualenv python-mastery-capstone-Sh3whZ1x-py3.14 in C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs
Using version ^26.5.1 for black
Using version ^9.0.1 for isort
Using version ^0.16.7 for ruff
Using version ^4.6.2 for pre-commit

Updating dependencies
Resolving dependencies... (1.2s)

Package operations: 18 installs, 0 updates, 0 removals

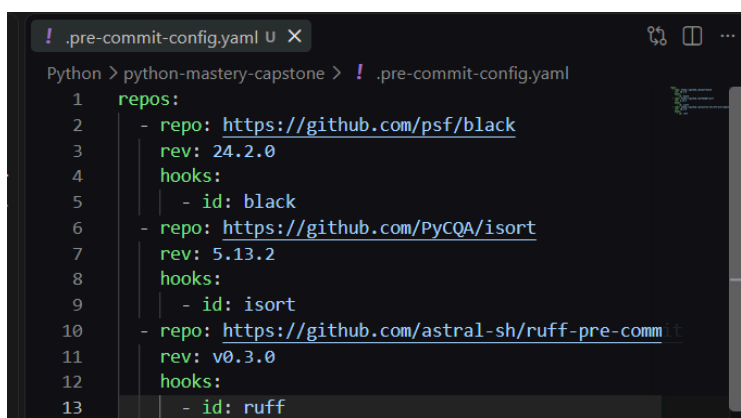
- Installing filelock (3.32.6)
- Installing distlib (0.4.3)
- Installing platformdirs (4.11.8)
- Installing python-discovery (1.6.0)
- Installing cfgv (3.5.0)
- Installing click (8.5.0)
- Installing mypy-extensions (1.1.0)
- Installing identify (2.6.19)
- Installing nodeenv (1.10.0)
- Installing packaging (26.3)
- Installing pathspec (1.1.1)
- Installing pytokens (0.4.1)
- Installing pyyaml (6.0.3)
- Installing virtualenv (21.7.9)
- Installing black (26.5.1)
- Installing isort (9.0.1)
- Installing ruff (0.16.7)
- Installing pre-commit (4.6.2)

Writing lock file
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Las librerías quedan registradas dentro del bloque en el archivo `pyproject.toml` y se genera el archivo `poetry.lock` para congelar las versiones exactas de cada paquete instalado y garantizar reproducibilidad en otros entornos.

- Configurar pre-commit; corregir infracciones PEP 8 iniciales

1. Configurar pre-commit: Creamos el archivo de especificación `.pre-commit-config.yaml` en la raíz del proyecto para vincular los repositorios oficiales de black, isort y ruff.



```
! .pre-commit-config.yaml U X
Python > python-mastery-capstone > ! .pre-commit-config.yaml
1 repos:
2   - repo: https://github.com/psf/black
3     rev: 24.2.0
4     hooks:
5       - id: black
6   - repo: https://github.com/PyCQA/isort
7     rev: 5.13.2
8     hooks:
9       - id: isort
10  - repo: https://github.com/astral-sh/ruff-pre-commit
11    rev: v0.3.0
12    hooks:
13      - id: ruff
```

2. Activamos los ganchos (hooks) dentro del directorio interno de Git para interceptar cada commit y auditar el código de forma automática antes de permitir guardar cambios en el historial (cada git commit).

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry
run pre-commit install
pre-commit installed at .git\hooks\pre-commit
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> █
```

3. Creamos la estructura de las carpetas:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> New-Item -ItemType Directory -Path "labs/day01_tooling" -Force

Directorio:
C:\Users\Quiryat\Documents\Python\python-mastery-capstone\labs

Mode                LastWriteTime         Length Name
----                -
d-----         11/09/2026 12:35 p. m.         day01_tooling

PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> █
```

4. Creamos un archivo de prueba `labs/day01_tooling/datos.json`, que representará una fuente de datos cruda, la cuál simula información recibida desde una API externa o un sistema de archivos:

```
PS C:\Ucd labs\Documents\Python\python-mastery-capstone>
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone\labs> cd day01_tooling
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone\labs\day01_tooling> code datos.j
son
```

Se definen cuatro registros dentro de una lista de diccionarios, cada uno con las claves id, nombre, precio y activo.

Casos de prueba diseñados intencionalmente:

- Casos exitosos: (Registros 1 y 2). Elementos con activo: true y precios numéricos válidos (1200.50 y 25.00).
- Casos de error de datos: (Registro 3). Elemento activo cuyo precio es un texto inválido ("precio inválido"), diseñado para probar la captura de excepciones (ValueError).
- Caso de filtrado: (Registro 4). Elemento inactivo (activo: false), diseñado para validar la lógica del control de flujo.

```
! .pre-commit-config.yaml | X  {} datos.json | X  script_json.py | X
Python > python-mastery-capstone > labs > day01_tooling > {} datos.json > ...
1  [
2  {"id": 1, "nombre": "Laptop", "precio": 1200.50, "activo": true},
3  {"id": 2, "nombre": "Mouse", "precio": 25.00, "activo": true},
4  {"id": 3, "nombre": "Teclado", "precio": "precio_invalido", "activo": true},
5  {"id": 4, "nombre": "Monitor", "precio": 300.00, "activo": false}
6  ]
```

5. Creamos el script para procesar el archivo de prueba `script_json.py`:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone\labs\day01_tooling> code script_
json.py
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone\labs\day01_tooling> █
```

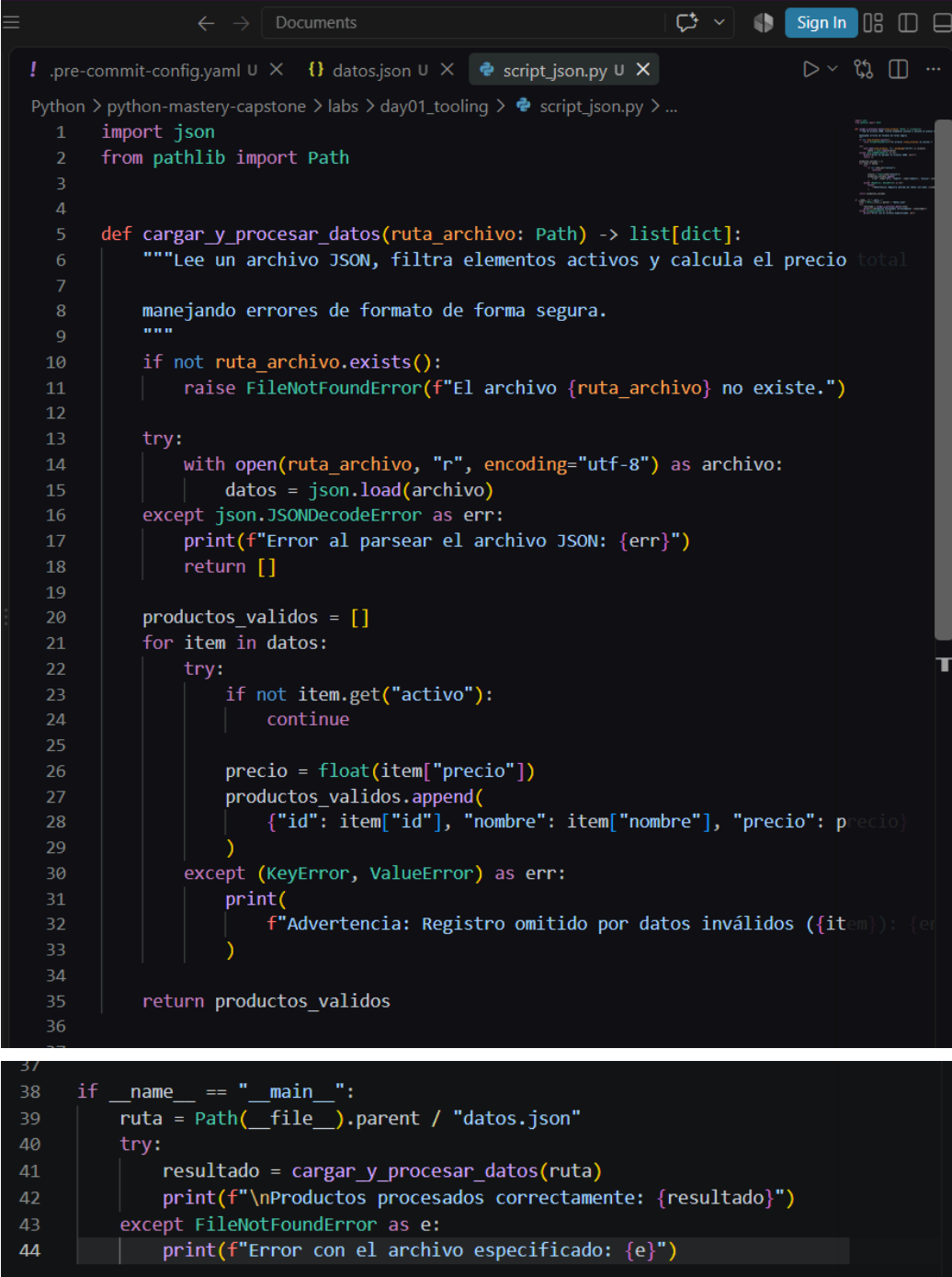
El script inicia con un módulo nativo para convertir texto JSON (línea 1) a estructuras de datos nativas de Python (listas y diccionarios). Permite manipular rutas de archivos de forma orientada a objetos y compatible entre distintos sistemas operativos (línea 2).

Continúa definiendo la función principal, firma con tipado estático (type hints) la cual recibe un objeto de tipo Path y retorna una lista de diccionarios (list[dict]), alineado al estándar del módulo del tipado (línea 5).

Después hace la apertura de archivo con administrador de contexto y control de excepciones haciendo una validación previa verificando si el archivo existe antes de intentar abrirlo (línea 10 y 11), with open garantiza que el archivo se cierre automáticamente en memoria al terminar de leerlo, incluso si ocurre un error, try-except json.JSONDecodeError captura específicamente errores de sintaxis en el archivo (Línea 13 - 18).

Para el control de flujo, el filtrado y la conversión de tipos usamos el bucle for e if not item.get("activo"), este filtra y descarta los productos inactivos mediante la instrucción continue, la conversión se hace con float(item["precio"]) intentando convertir el precio a número decimal, seguido tenemos el manejo puntual de excepciones except (KeyError, ValueError), el primero por si a algún registro le falta una clave requerida y el segundo por si no se puede convertir a número, cómo ocurre en el registro 3. Ahora, en lugar de detener la ejecución de todo el programa, el script ignora el registro corrupto, imprime una advertencia y continúa procesando el resto (línea 20 - 33).

Por último tenemos el punto de entrada principal, con Path(__file__).parent / "datos.json" construimos la ruta absoluta hacia datos.json relativa a la ubicación del script, evitando errores de carpeta cuando se ejecuta desde la raíz del proyecto. También tenemos una excepción específica (FileNotFoundError), se utiliza esta excepción explícita en lugar de una genérica (except Exception) para cumplir con la regla BLE001 de Ruff y las buenas prácticas de diseño PEP 8 / PEP 20.



```
! .pre-commit-config.yaml U X {} datos.json U X script_json.py U X
Python > python-mastery-capstone > labs > day01_tooling > script_json.py > ...
1 import json
2 from pathlib import Path
3
4
5 def cargar_y_procesar_datos(ruta_archivo: Path) -> list[dict]:
6     """Lee un archivo JSON, filtra elementos activos y calcula el precio total
7
8     manejando errores de formato de forma segura.
9     """
10
11     if not ruta_archivo.exists():
12         raise FileNotFoundError(f"El archivo {ruta_archivo} no existe.")
13
14     try:
15         with open(ruta_archivo, "r", encoding="utf-8") as archivo:
16             datos = json.load(archivo)
17     except json.JSONDecodeError as err:
18         print(f"Error al parsear el archivo JSON: {err}")
19         return []
20
21     productos_validos = []
22     for item in datos:
23         try:
24             if not item.get("activo"):
25                 continue
26
27             precio = float(item["precio"])
28             productos_validos.append(
29                 {"id": item["id"], "nombre": item["nombre"], "precio": precio}
30             )
31         except (KeyError, ValueError) as err:
32             print(
33                 f"Advertencia: Registro omitido por datos inválidos ({item}): {err}"
34             )
35
36     return productos_validos
37
38 if __name__ == "__main__":
39     ruta = Path(__file__).parent / "datos.json"
40     try:
41         resultado = cargar_y_procesar_datos(ruta)
42         print(f"\nProductos procesados correctamente: {resultado}")
43     except FileNotFoundError as e:
44         print(f"Error con el archivo especificado: {e}")
```

6. Cuando ejecutamos el script observamos los datos filtrados correctamente y las advertencias de los registros omitidos:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run python labs/day01_tooling/script_json.py
Advertencia: Registro omitido por datos inválidos ({'id': 3, 'nombre': 'Teclado', 'precio': 'precio_invalido', 'activo': True}): could not convert string to float: 'precio_invalido'

Productos procesados correctamente: [{'id': 1, 'nombre': 'Laptop', 'precio': 1200.5}, {'id': 2, 'nombre': 'Mouse', 'precio': 25.0}]
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Intentamos hacer un commit para subir nuestros cambios a Github creando una nueva rama para este ejercicio.

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git checkout -b feature/day-01-environment
Switched to a new branch 'feature/day-01-environment'
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>

PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git add .
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git commit -m "lab(day01): Configuración de pre-commit y script de prueba"
[INFO] Initializing environment for https://github.com/psf/black.
[INFO] Initializing environment for https://github.com/PyCQA/isort.
[WARNING] repo `https://github.com/PyCQA/isort` uses deprecated stage names (commit, merge-commit, push) which will be removed in a future version. Hint: often `pre-commit autoupdate --repo https://github.com/PyCQA/isort` will fix this. if it does not -- consider reporting an issue to that repo.
[INFO] Initializing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Installing environment for https://github.com/psf/black.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/PyCQA/isort.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
[INFO] Installing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
black.....Failed
- hook id: black
- files were modified by this hook

reformatted labs\day01_tooling\script_json.py

All done! \u2728 \U0001f370 \u2728
1 file reformatted.

isort.....Passed
ruff.....Passed
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

7. Corrección de infracciones:

- Corrección de reglas de lintado (Ruff): El linter de Ruff detectó la infracción BLE001 (Do not catch blind exception) por el uso de except Exception: en el script. Se corrigió especificando la excepción exacta except FileNotFoundError:, cumpliendo con la guía de diseño PEP 8 y PEP 20 sobre manejo explícito de errores.
- Formateo automático de estilo (Black): Al ejecutar el primer git commit, black devolvió el estado Failed porque el código no cumplía estrictamente con la indentación y espacios de PEP 8. black reestructuró automáticamente el archivo script_json.py.

- Verificación y paso de pruebas: Tras preparar los cambios formateados (git add .) y ejecutar git commit, todos los hooks (black, isort, ruff) arrojaron el estado Passed.
8. Dado que el archivo fue actualizado, agregamos el archivo formateado de nuevo a Git y ejecutamos el commit: El archivo quedó alineado al estándar oficial de estilo PEP 8 y pasó todas las reglas de calidad.

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git add .

PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git commit -m "lab(day01): configuración de pre-commit y script de prueba modificado"
black.....
..Passed
isort.....
..Passed
ruff.....
..Passed
[feature/day-01-environment 8db05af] lab(day01): configuración de pre-commit y script de prueba modificado
5 files changed, 541 insertions(+)
 create mode 100644 .pre-commit-config.yaml
 create mode 100644 labs/day01_tooling/datos.json
 create mode 100644 labs/day01_tooling/script_json.py
 create mode 100644 poetry.lock
 create mode 100644 pyproject.toml
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Ahora todos dan el check Passed (verde).

Módulo 2

- Script que lee JSON, filtra/agrega datos y maneja errores de archivo/formato

1. Filtrado y agregación de datos: La función `json.load(archivo)` parsea el texto del archivo `datos.json` y lo transforma en una lista de diccionarios de Python y se utiliza el administrador de contexto `with open(ruta_archivo, "r", encoding="utf-8")` as `archivo`: para garantizar la liberación del archivo de la memoria al finalizar la lectura.

El filtrado se realiza por estado y estructura con `case {"activo": False}`: que detecta y descarta de inmediato los registros desactivados; el filtrado por validación que descarta registros que no cumplen el patrón `regex` del SKU o cuya conversión de precio falla; y la agregación selectiva con `productos_validos.append(valido)` que construye de forma progresiva una lista limpia que únicamente almacena registros que superaron todas las validaciones de tipo y formato.

2. Manejo de errores de archivo: La verificación preventiva con `if not ruta_archivo.exists()`: evalúa si el archivo existe físicamente antes de intentar abrirlo; y el control de excepciones de lectura lanza y captura de manera explícita la excepción `FileNotFoundError` en el punto de entrada (`__main__`), informando la causa del error sin romper la ejecución del script.

3. Manejo de errores de formato y esquema: El bloque try-except (ValueError, TypeError) sobre float(precio_raw) procesa valores que no se pueden convertir a números decimales; La clase ErrorDatoInvalido aísla los fallos por registro, al capturarla dentro del bucle for, el script imprime un mensaje de advertencia sobre el registro dañado y continúa procesando el resto de elementos sin colapsar; el patrón case _: del match-case atrapa cualquier objeto JSON que carezca de los campos mínimos esperados (sku, nombre, precio, activo).

```
1  import json
2  import re
3  from pathlib import Path
4
5  # Expresión regular para validar el código SKU (3 letras - 3 números)
6  PATRON_SKU = r"^[A-Z]{3}-\d{3}$"
7
8
9  class ErrorDatoInvalido(Exception):
10 |     """Excepción para errores en campos del JSON."""
11
12
13  def procesar_registro(item: dict) -> dict:
14 |     """Aplica Pattern Matching y Expresiones Regulares para validar cada registro."""
15 |     match item:
16 |         case {
17 |             "sku": sku,
18 |             "nombre": nombre,
19 |             "precio": precio_raw,
20 |             "activo": True,
21 |         }:
22 |             if not re.match(PATRON_SKU, sku):
23 |                 raise ErrorDatoInvalido(f"SKU '{sku}' no cumple el formato 'AAA-000'")
24
25 |             try:
26 |                 precio = float(precio_raw)
27 |                 if precio <= 0:
28 |                     raise ValueError("El precio debe ser positivo")
29 |             except (ValueError, TypeError) as e:
30 |                 raise ErrorDatoInvalido(f"Precio inválido ({precio_raw}): {e}")
31
32 |             return {"sku": sku, "nombre": nombre, "precio": precio}
33
34 |         case {"activo": False}:
35 |             raise ErrorDatoInvalido("Registro inactivo")
36
```

```
37         case _:
38             raise ErrorDatoInvalido("Estructura de registro no compatible o incompleta")
39
40
41 def cargar_y_procesar_datos(ruta_archivo: Path) -> list[dict]:
42     if not ruta_archivo.exists():
43         raise FileNotFoundError(f"El archivo {ruta_archivo} no existe.")
44
45     with open(ruta_archivo, "r", encoding="utf-8") as archivo:
46         datos = json.load(archivo)
47
48     productos_validos = []
49     for item in datos:
50         try:
51             valido = procesar_registro(item)
52             productos_validos.append(valido)
53         except ErrorDatoInvalido as err:
54             print(f"Registro omitido: {err}")
55
56     return productos_validos
57
58
59 if __name__ == "__main__":
60     ruta = Path(__file__).parent / "datos.json"
61     try:
62         resultado = cargar_y_procesar_datos(ruta)
63         print(f"\nProductos procesados correctamente: {resultado}")
64     except FileNotFoundError as e:
65         print(f"Error con el archivo especificado: {e}")
66
```

LAP-001 y MOU-002 pasan la validación de pattern matching ya que su SKU coincide con el patrón de la expresión regular y tienen precios decimales válidos.

TECLADO-99 es capturado por el pattern matching por tener activo:true, pero falla en la validación por expresiones regulares al no coincidir con el formato AAA-000 y se lanza y captura la excepción personalizada ErrorDatoInvalido.

MON-004 es evaluado por el pattern matching case {"activo": false} antes de procesar otros campos y lanza ErrorDatoInvalido("Registro inactivo"), omitiendo limpiamente.

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run python labs/day01_tooling/script_json.py
Registro omitido: SKU 'TECLADO-99' no cumple el formato 'AAA-000'
Registro omitido: Registro inactivo

Productos procesados correctamente: [{'sku': 'LAP-001', 'nombre': 'Laptop', 'precio': 1200.5}, {'sku': 'MOU-002', 'nombre': 'Mouse', 'precio': 25.0}]
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> 
```

Anexo

PEP 8 Guía de estilo para código Python.

- <https://peps.python.org/pep-0008/#introduction>

PEP 20 El Zen de Python.

- <https://peps.python.org/pep-0020/>

Lo bello es mejor que lo feo.

Lo explícito es mejor que lo implícito.

Lo simple es mejor que lo complejo.

Lo complejo es mejor que lo complicado.

Lo plano es mejor que lo anidado.

Lo disperso es mejor que lo denso.

La legibilidad cuenta.

Los casos especiales no son lo suficientemente especiales como para romper las reglas.

Aunque la practicidad prevalece sobre la pureza.

Los errores nunca deben pasar en silencio.

A menos que se silencien explícitamente.

Ante la ambigüedad, rechaza la tentación de adivinar.

Debería haber una —y preferiblemente sólo una— manera obvia de hacerlo.

Aunque esa manera puede no ser obvia al principio, a menos que seas holandés.

Ahora es mejor que nunca.

Aunque nunca suele ser mejor que *justo* ahora.

Si la implementación es difícil de explicar, es una mala idea.

Si la implementación es fácil de explicar, puede que sea una buena idea.

Los espacios de nombres son una idea fantástica: ¡hagamos más de ellos!